NFH-Kuza-Google — Unified Domain-Agnostic Architecture

Single platform for Local Services (Agri/Rural) + Health Services (TIBU) + future domains

Domains: **local-services** + **health-services**

Version: 2.0

Date: April 2026

Status: Draft

1. Project Overview

NFH-Kuza-Google is a **unified service orchestration platform** that connects *seekers* (farmers, patients, households) with *providers* (agripreneurs, clinicians, facilities, aggregators) across multiple domains through WhatsApp, a lightweight web application, and low-connectivity channels (SMS/USSD, Mobile Lite). The platform is built on the **Beckn open network protocol**, enabling decentralized discovery and fulfillment of services.

The same underlying flow engine, architecture, technology stack, and data infrastructure serve two live domains today, with a design that admits additional domains without structural change:

- **Local Services / Agri** (Beckn domain code: `local-services`) — Livelihood, Household, Market, and Advisory flows for rural households.
- **Health Services / TIBU** (Beckn domain code: `health-services`) — TB screening, treatment, supervision, and post-TB care for Kenya's National TB Programme.

> **Design Principle**
>
> 100% of seeker interaction occurs through channel-agnostic messaging (primarily WhatsApp) — no app installation required. Providers use a lightweight Ionic Angular web app opened via WhatsApp button links, with an optional Mobile Lite interface for field workers (CHVs, CHPs, extension officers) in low-bandwidth conditions.

Service Domains — Agri (local-services)

- **Livelihood Services** — agricultural inputs, equipment rental, veterinary services, soil testing, etc.
- **Household Services** — plumbing, electrical, cleaning, construction, etc.
- **Market Produce** — connecting farmers with aggregators and buyers (reversed economics: buyer pays seeker)
- **Advisory** — agricultural content and guidance delivered as media (no order lifecycle)

Service Domains — Health (health-services)

- **Screening & Referral** — symptom triage, referral to nearest facility with the right capability (e.g., GeneXpert, X-ray)
- **Treatment Adherence** — medication reminders, directly-observed therapy (DOT), supervision by CHP/CHV
- **Lab & Diagnostics** — sputum collection, GeneXpert, culture & DST bookings
- **Post-TB Care & Advisory** — PTLD monitoring, nutrition, TB-awareness education

1.5 Unified Flow Engine

The central abstraction of the platform is a **domain-agnostic 7-step Flow Engine**. Every transactional interaction — whether agri or health — is modelled as the same sequence of states. This is what makes the architecture truly unified: verticals differ in schema and vocabulary, but the engine that orchestrates them is identical.

> **The 7 Steps**
>
> **Need → Search → Select → Confirm → Deliver → Record → Outcome**

- **Need** — a seeker has a need (farmer needs vaccination; patient has persistent cough). User trigger; no Beckn call.
- **Search** — identify who can help, matching capability + location. Beckn: `discover` / `on_discover`.
- **Select** — match to best provider, including quote / booking. Beckn: `select` / `on_select` + `init` / `on_init`.
- **Confirm** — both sides agree, binding contract created. Beckn: `confirm` / `on_confirm`.
- **Deliver** — service rendered (vaccination given; sputum collected). Beckn: `status` updates (`PROVIDER_ACCEPTED`, `IN_PROGRESS`, `COMPLETED`).
- **Record** — data captured (visit notes, clinical record, payment record). Beckn: `update` + `status`.
- **Outcome** — tracked to result (farmer feedback, treatment success, cure confirmed). Beckn: `rate` / `on_rate` plus custom outcome events (e.g., `HEALTH_OUTCOME_RECORDED`). Can be immediate or longitudinal.

Generalized Actor Model

Specific titles (farmer, patient, agripreneur, clinician) are expressions of four abstract roles. A single individual may play multiple roles across different flows — a CHP is a *Provider* for screening but a *Seeker* for supervision.

- **Seeker** — any individual or household needing service (farmer, patient, household, CHV, CHP).
- **Provider** — anyone delivering service (agripreneur, vet, clinician, lab, agro-vet, training facility).
- **Coordinator** — supervisors / oversight (SCTLC, CTLC, extension officer, area manager).
- **Validator** — lab, audit, quality assurance.
- **Sponsor** — funders, partners, government, NGOs.

Domain & Vertical Hierarchy

```
Platform (Kuza Flow Engine)
  └── Domain (e.g., local-services, health-services)
        └── Vertical (e.g., poultry, crops, TB, HIV, malaria)
              └── Flow Template (e.g., vaccination-booking, screening-referral)
```

Each Beckn domain code maps to one platform Domain. Verticals are sub-classifications within a domain. Flow templates are reusable parameterised patterns instantiated by the Flow Engine.

Capability-Based Matching

Providers are matched by **service area + structured capability tags**, not category alone:

- **Agri**: `services_offered` (vaccination, brooding, AI, soil testing, etc.)
- **Health**: `capabilities` (GeneXpert, X-ray, spirometry, DRTB-certified clinician)

Both stored as structured tags on the Provider profile, queried in `discover`.

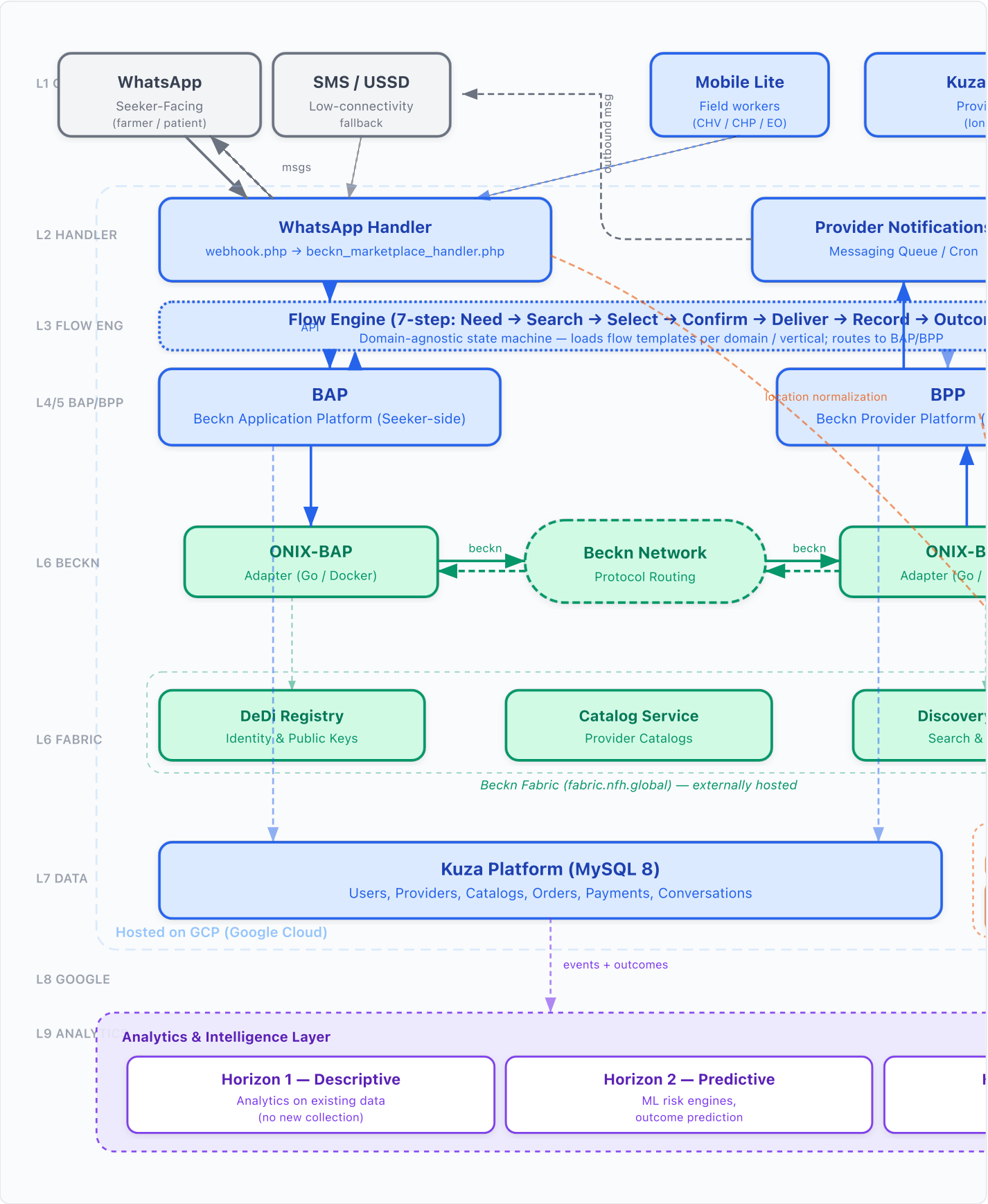
Outcome Recording

Beyond service status `COMPLETED`, the engine captures an explicit **Outcome** phase:

- **Agri outcomes**: harvest yield (later season), income earned, repeat business, farmer feedback.
- **Health outcomes**: cure confirmed, treatment success, prevented mortality, PTLD detected.
- Outcomes can be captured *immediately* (feedback at close) or *longitudinally* (months later), feeding back into the Analytics layer for ML training.

2. Architecture Diagram

Kuza-built components · Beckn infrastructure · Google services · Channels / external · Analytics & Intelligence



3. Architecture Layers

The architecture is organised into **nine layers**. Layers 1-8 run on every deployment; Layer 9 (Analytics & Intelligence) is additive and operates on the data emitted by the other layers.

3.1 Layer 1: Frontend Channels

The platform is **channel-agnostic**. The same flow can be driven over any supported channel; channel choice is a function of the seeker's device and connectivity, not of the domain.

WhatsApp (Seeker-Facing, Primary)

WhatsApp is the primary seeker-facing interface. The bulk of seeker interactions — discovery, provider selection, confirmation, payment declaration, feedback, treatment reminders — occur through WhatsApp messages.

- **Interactive buttons** for small binary/ternary choices (Accept/Reject, Yes/No, payment method selection)
- **Numbered text lists** for longer option sets (provider lists, service categories, sub-categories)
- **Free text input** for location, quantity, symptoms, and other open-ended fields
- Connected via **WhatsApp Cloud API**

SMS / USSD (Low-Connectivity Fallback)

For seekers without smartphones or data, the same flow engine is reachable via SMS and USSD shortcodes. Message structure is simplified (numeric menus, short prompts) but the underlying 7-step sequence is preserved.

Mobile Lite (Field Workers)

A **TIBUlite-style** lightweight mobile client for field workers (CHVs, CHPs, extension officers) operating in low-bandwidth areas. Supports offline queueing of Deliver / Record steps with sync-on-reconnect.

Kuza Web App (Provider-Facing)

A lightweight Ionic Angular web application opened from WhatsApp button links — no native app installation required. Providers (agripreneurs, clinicians, facilities) use it to:

- Manage their service catalog / capability profile (add/edit services, set pricing, declare capabilities such as GeneXpert, vaccination, etc.)
- View and accept/reject incoming orders / referrals
- Update service status (in-progress, completed)
- Capture clinical / visit records during the Record step
- Manage their profile and service areas

3.2 Layer 2: Channel Handlers

> **Design Constraint**
> The existing `webhook.php` remains UNTOUCHED. Only routing logic is added to branch marketplace conversations to the new `beckn_marketplace_handler.php` based on trigger words or message IDs.

- **File:** `beckn_marketplace_handler.php` (new PHP file on Kuza Platform)
- **State machine:** manages conversational flow for the 4 marketplace domains (Livelihood, Household, Market, Advisory)
- **Translates** WhatsApp interactions into BAP API calls
- **Receives** async callbacks from BAP and formats WhatsApp response messages
- **Own tables:** conversation state, session tracking — separate from existing poultry chatbot tables (`chatbot_nodes`, `chatbot_transitions`)
- **Provider notifications** are triggered via a messaging queue/cron job, NOT through `webhook.php`

3.3 Layer 3: Flow Engine

The Flow Engine sits **above BAP/BPP** and **below the Channel Handlers**. It is the domain-agnostic state machine that implements the 7-step lifecycle (Need → Search → Select → Confirm → Deliver → Record → Outcome) and decides which Beckn call (if any) each step translates into.

- **Flow templates** per domain / vertical (`vaccination-booking`, `screening-referral`, `market-sell-produce`, `treatment-adherence`) are loaded at runtime.
- **Transaction state** is persisted per seeker session (`flow_instance_id`) and correlated with the Beckn transaction id.
- **Routing** — each step may produce a BAP call, a BPP-originated event, or a pure UX transition (e.g., the Need step has no network call).
- **Progressive disclosure** — the same template can render differently by actor role (seeker vs. coordinator vs. validator).
- **Optional adoption** — the engine is not required for read-only analytics; existing non-flow interactions (one-off surveys, content pulls) continue to operate independently.

3.4 Layer 4: BAP (Beckn Application Platform)

The BAP is the seeker-side Beckn participant (e.g., farmer, patient). It acts as the bridge between the Flow Engine and the Beckn network.

- Receives translated requests from the WhatsApp Handler
- Constructs Beckn v2 protocol messages: `discover`, `select`, `init`, `confirm`, `status`, `cancel`, `rate`

- Sends protocol messages through the ONIX-BAP adapter
- Receives async callbacks (`on_discover` , `on_select` , `on_init` , `on_confirm` , `on_status` , `on_cancel` , `on_rate`)
- Pushes results back to WhatsApp Handler for farmer delivery
- Hosted by Kuza on GCP

3.5 Layer 5: BPP (Beckn Provider Platform)

The BPP is the provider-side Beckn participant. It manages the provider catalog, order lifecycle, and serves the provider web interface.

- Receives Beckn protocol requests from the network (via ONIX-BPP adapter)
- Connects to Kuza Platform to fetch provider catalogs, service areas, and pricing
- Manages order lifecycle on the provider side (create, accept, update status, complete)
- Sends async Beckn responses back through ONIX-BPP adapter
- Serves the **Kuza Web App** (provider interface) for catalog and order management
- Hosted by Kuza on GCP

3.6 Layer 6: Beckn Network Infrastructure

All Beckn network components are external to Kuza infrastructure, provided by Beckn Fabric.

ONIX Adapters

- Plugin-based Go servers running the `fidedocker/onix-adapter` Docker image
- Two instances: ONIX-BAP (seeker side — farmer / patient) and ONIX-BPP (provider side — agripreneur / clinician / facility)
- Handle message signing, signature validation, schema validation, and network routing
- Hosted on GCP as Docker containers

DeDi Registry

- Endpoint: `fabric.nfh.global/registry/dedi`
- Stores participant identity, public keys, and callback URIs
- Used for signature validation and dynamic routing between BAP and BPP

Catalog Service

- Endpoint: `fabric.nfh.global/beckn/catalog`
- BPP publishes provider catalogs here

- Queried by Discovery Service during `discover` requests

Discovery Service

- Handles `discover` requests from BAP
- Queries Catalog Service for matching providers
- Returns results to BAP via `on_discover` callback

3.7 Layer 7: Kuza Platform / Data Foundation

MySQL 8 (with a PostgreSQL migration under way in `kuza-next`) on GCP serves as the single source of truth. The Beckn network is stateless — all persistent data lives here.

> **Sovereign Data & Compliance**
>
> Seeker / patient / farmer data stays in country. Records are persisted with a full audit trail (status transitions, who updated, when). For the health domain, the record schema is designed to be **FHIR-compatible** so clinical records can be exported into national health information systems without reshaping.

Existing Tables Used

- `user` — unified seeker / provider identity (agripreneurs, farmers, clinicians, patients, CHPs)
- Partner and organization data
- Content library (for Advisory domain, agri & health)

New Tables Required

Table	Purpose
`beckn_provider_catalog`	Services offered by providers, with pricing per service
`beckn_provider_service_areas`	County + Sub-County mappings for provider coverage areas
`beckn_subcategory_master`	Dynamic taxonomy of service sub-categories
`beckn_orders`	Order records with full lifecycle tracking
`beckn_order_status_history`	Audit trail of status transitions per order
`beckn_payments`	Payment records (farmer-side and provider-side)
`beckn_feedback`	Feedback and ratings from both parties
`beckn_conversation_state`	Marketplace conversation state machine data

Table	Purpose
`beckn_farmer_profile`	One-time farmer profile fields (location, preferences)

3.8 Layer 8: Google Cloud Services

Google contributes to the platform across **three distinct service tiers**: cloud infrastructure for hosting, content APIs for serving curated domain content, and AI interfaces for intelligent processing. Each is consumed at a different point in the pipeline.

3.8.1 Google Cloud Platform (GCP) — Hosting Infrastructure

The entire Kuza-built stack is deployed on GCP. This is the foundational layer on which everything Kuza builds runs.

- **Compute:** hosts the WhatsApp Handler, BAP application, BPP application, Kuza Web App, ONIX adapters (Docker)
- **Database:** hosts MySQL 8 (the Kuza Platform data store)
- **Networking:** public endpoints, TLS termination, load balancing for BAP / BPP callbacks
- **Co-location benefit:** Kuza application code and Google Content APIs / AI interfaces are co-located on the same cloud, minimising latency for in-flow API calls

3.8.2 Google Content APIs — Curated Domain Content

Google publishes a set of **curated content APIs per domain** (agri, health, market, finance, etc.) that serve *informational / educational* content to farmers via the chatbot. These are **separate from Beckn transactional catalog data** — they don't list providers or services for sale, they deliver knowledge.

- **Primary consumer:** Advisory flow — the BPP calls Google Content APIs (and Google AI) during `on_discover` processing to source advisory material. Content may include curated articles, videos, audio, infographics, or AI-generated summaries.
- **Secondary consumers:** Livelihood / Household / Market flows — these can enrich a transaction with supplementary content (e.g., "Best practices for vaccinating broilers" attached to a Vaccination order; "Egg storage tips" attached to a Market produce sale). Used optionally to add value beyond the transaction.
- **Domain coverage:** agri, health, market, finance, education — matched to Kuza's flow categories. Each domain has its own taxonomy of categories and sub-categories that align with the master data taxonomy in Kuza Platform.
- **Delivery format:** APIs return content payloads (text, media URLs, structured guides) that the BPP wraps into the Beckn `on_discover` callback for Advisory, or surfaces as supplementary messages in the chatbot for other flows.

- **Called from:** BPP application logic (during request processing) and optionally from the WhatsApp Handler for inline content.

3.8.3 Google AI Interfaces — Intelligent Processing

Distinct from content APIs, the AI interfaces provide *computational* services that process input and return derived results.

- **Location normalization (all flows):** Farmer enters free-text location (village / ward / county) → PHP handler sends text to Google AI → AI maps to structured County + Sub-County → PHP handler sends confirmation message to farmer on WhatsApp → farmer confirms or corrects.
- **Advisory content generation (Advisory flow):** When curated content from the Content APIs needs personalisation, summarisation, or translation, the BPP combines Content API output with farmer context and passes it through the Google AI interface to produce the final response delivered to the farmer.
- **Future:** intent extraction from free-text farmer queries, language translation (English ↔ Swahili), sentiment analysis on feedback.
- **Called from:** PHP handler (location normalization) and BPP (advisory generation, future intent extraction).

> **Content APIs vs AI Interfaces — how they relate:** Content APIs serve *what we know* (curated knowledge bases per domain). AI Interfaces serve *what we infer* (normalising, generating, translating). The Advisory flow typically uses both: Content API supplies the source material, AI Interface tailors it for the specific farmer.

3.9 Layer 9: Analytics & Intelligence

Layer 9 sits *beside* the transactional stack and consumes the event / outcome stream that every flow emits. Because outcomes are a first-class concept in the Flow Engine, analytics and ML have rich training signals without needing bespoke collection pipelines.

Deployment is staged along three **horizons**:

- **Horizon 1 — Descriptive analytics on existing data**: dashboards, cohort performance, funnel conversion, provider scorecards — no new collection.
- **Horizon 2 — Predictive ML**: farmer risk scoring (mortality, profitability), patient adherence risk, outcome prediction, facility capability-gap detection.
- **Horizon 3 — Pioneering surveillance**: multi-domain, longitudinal studies (e.g., linking nutrition + TB outcomes, weather + animal disease). This is only feasible *because* the platform is domain-agnostic.

> **Flow Engine is optional until proven**

> Horizon 1 analytics can run on any existing Kuza data without requiring the Flow Engine. Horizon 2 and 3 become meaningful as flow-emitted outcome data accumulates.

4. Key Data Flows

Flow A: Farmer Discovers and Books a Service (Livelihood / Household)

```
Farmer (WhatsApp): Selects category --> service --> enters quantity
  --> webhook.php --> beckn_marketplace_handler.php
    Farmer types location (free text, e.g. "Naivasha")
    --> PHP Handler calls Google AI for location normalization
    <-- Google AI returns {county: "Nakuru", subCounty: "Naivasha"}
    --> WhatsApp: "Nakuru County, Naivasha Sub-County. Correct?" [Yes] [No]
    Farmer confirms [Yes]
    --> BAP Client --> ONIX-BAP --> [Beckn Network / Discovery Service] --> ONIX-BPP -->
BPP
      --> Kuza Platform (query providers by location + category + subCategory)
    <-- BPP --> ONIX-BPP --> [Beckn Network] --> ONIX-BAP --> BAP Client
  <-- beckn_marketplace_handler.php --> WhatsApp (list of providers with prices)

(continues through select / init / confirm / status / update / cancel / rate / support —
 see Section 4.6 for the complete Beckn endpoint mapping covering the full order
lifecycle.)
```

Flow B: Provider Receives and Accepts Order

```
Order confirmed in Beckn
  --> BPP --> Kuza Platform (create order record)
  --> Messaging Queue / Cron --> WhatsApp Cloud API --> Provider WhatsApp
    Provider taps [Accept] button in WhatsApp
      --> webhook.php --> provider routing --> BPP --> Beckn update
    Provider taps [View Details] link
      --> Opens Kuza Web App in browser --> reads from Kuza Platform
```

Flow C: Provider Completes Service, Farmer Confirms

```
Provider (Kuza Web App):
  --> Marks service complete, enters payment details, notes
  --> BPP --> Beckn status update --> ONIX-BAP --> BAP
```

```
  --> beckn_marketplace_handler.php --> WhatsApp to Farmer
    "Was the service completed?" [Yes] [No]
  Farmer taps [Yes]
  --> webhook.php --> beckn_marketplace_handler.php
    --> "How did you pay?" [Cash] [M-Pesa]
  Farmer selects payment method
  --> Order marked FARMER_CONFIRMED --> CLOSED
```

Flow D: Farmer Gets Advisory Content (Google Content APIs + AI)

```
Farmer (WhatsApp): Selects Advisory --> Domain (Agri/Health/Market/Finance) --> Category
--> Sub-category
  --> webhook.php --> beckn_marketplace_handler.php
    --> BAP Client --> ONIX-BAP --> [Beckn Network] --> ONIX-BPP --> BPP

      ┌── BPP processing ──────────────────────────────────────┐
      | 1. BPP calls Google Content API for {domain, category,     |
      |    sub-category} --> returns curated content payload       |
      |    (articles, video URLs, guides)                          |
      | 2. (Optional) BPP calls Google AI Interface to             |
      |    personalise / summarise / translate using farmer        |
      |    context (location, enterprise, language)                |
      | 3. BPP composes the final advisory response                |
      └────────────────────────────────────────────────────────────┘

    <-- BPP --> ONIX-BPP --> [Beckn Network] --> ONIX-BAP --> BAP Client
  <-- beckn_marketplace_handler.php --> WhatsApp (curated/AI-tailored advisory)

Google Content APIs supply the source material; Google AI tailors it.
Advisory is free, no order lifecycle. Content delivered immediately via on_discover.
```

Domain-agnostic: this flow works for agricultural advisory AND health advisory (e.g., TB awareness, post-TB care education, nutrition counselling). Only the content taxonomy and the Google Content API endpoint change.

Flow E: Market Produce (Farmer Sells)

```
Same flow as A but with reversed economics:
- Farmer has produce to sell (quantity + type)
- discover returns aggregators/buyers with buying prices
- Aggregator collects produce from farmer
- Aggregator pays farmer (not farmer pays provider)
- Both parties confirm and provide feedback
```

```
(continues through the same full Beckn lifecycle —
 see Section 4.6 for the complete endpoint mapping including cancellation & feedback.)
```

4.6 Complete Beckn Endpoint Mapping

This section maps every user-facing step in each flow to the corresponding Beckn v2 protocol endpoint, covering the **full order lifecycle** — discovery, selection, initialization, confirmation, fulfilment status, in-flight updates, cancellation, rating/feedback, and support.

Beckn endpoints reference:

- `discover` / `on_discover` — find services/providers in the network
- `select` / `on_select` — request a priced quote for a specific item
- `init` / `on_init` — submit billing & fulfilment details, receive draft order
- `confirm` / `on_confirm` — create the binding order
- `status` / `on_status` — query or push the current state of the contract
- `update` / `on_update` — mutate an existing contract (e.g. reschedule, report payment, add notes)
- `cancel` / `on_cancel` — cancel an order (before or after provider acceptance)
- `rate` / `on_rate` — submit ratings / feedback on provider or farmer
- `support` / `on_support` — request customer support details

> **Conventions used in the tables below:**
>
> - **BAP → BPP** means the call originates at the BAP (farmer side) and is routed through ONIX / Beckn Network to the BPP (provider side). The corresponding `on_*` callback travels in the opposite direction.
> - **status** calls can be *pull* (BAP → BPP to query) or *push* (BPP → BAP as a notification). Both are used.
> - Status values referenced come from `LocalServiceFulfillmentAttributes.serviceStatus`: `ORDER_PLACED`, `PROVIDER_ACCEPTED`, `PROVIDER_REJECTED`, `IN_PROGRESS`, `COMPLETED`, `FARMER_CONFIRMED`, `CANCELLED`, `CLOSED`.

4.6.1 Livelihood Services — Complete Lifecycle

Example: Poultry vaccination, feed supply, brooding, incubation, trades & skills.

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
Discovery	Farmer picks category, sub-category, service, quantity, location, preferred date	`discover`	BAP → BPP	Intent: category=Livelihood-Services, subCategory, quantityRequired, farmerLocation, scheduledDate
	System returns providers matching service area + service	`on_discover`	BPP → BAP	Catalog with providers, items (LivelihoodItemAttributes), offers (LocalServiceOfferAttributes) filtered by service area
Selection	Farmer selects one provider from the list	`select`	BAP → BPP	providerId, itemId, offerId, quantityRequired
	BPP returns final quote + cancellation policy	`on_select`	BPP → BAP	Final priceAmount (unit price × quantity), cancellationPolicy, confirmed offer terms
Initialization	Farmer provides contact details (auto from WhatsApp profile) and scheduled date	`init`	BAP → BPP	farmer contact, farmerLocation, scheduledDate, billing reference
	BPP returns draft order with payment terms	`on_init`	BPP → BAP	Draft order, payment terms (acceptedPaymentMethods: CASH/MPESA), total
Confirmation	Farmer taps [Confirm] on WhatsApp summary	`confirm`	BAP → BPP	Confirmation with consent flag
	Order created, provider notified	`on_confirm`	BPP → BAP	orderId, provider details, serviceStatus: ORDER_PLACED
Provider Response	Provider taps [Accept] in WhatsApp notification	`status` (push)	BPP → BAP	serviceStatus: PROVIDER_ACCEPTED, confirmedVisitTime
	Farmer receives "Provider accepted. Visit on <date>. Contact: <phone>"	*(push notification, no new call)*	—	Farmer notified on WhatsApp

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
	Provider taps [Reject] instead	`status` (push)	BPP → BAP	serviceStatus: PROVIDER_REJECTED → CLOSED; farmer re-presented with alternative providers
	Farmer checks order status manually	`status` (pull)	BAP → BPP	Query by orderId; returns `on_status` with current state
Mid-flight Changes	Provider or farmer changes date, quantity, time slot	`update`	BAP ↔ BPP (either side)	Updated scheduledDate / quantityRequired / confirmedVisitTime
	Other side confirms the change	`on_update`	Opposite direction	Revised order
Service Execution	Provider marks service in progress (via Kuza Web App)	`status` (push)	BPP → BAP	serviceStatus: IN_PROGRESS
	Provider completes service: enters actualServiceDate, payment method, amount received, notes, follow-up flag	`update` + `status`	BPP → BAP	serviceStatus: COMPLETED; providerAmountReceived, providerPaymentReceived: true, providerNotes, followUpRequired; on_update then on_status
Farmer Confirmation	Farmer receives "Was the service completed?" [Yes] [No]	*(WhatsApp; no Beckn call yet)*	—	—
	Farmer taps [Yes] → "How did you pay?" [Cash] [M-Pesa]	`update`	BAP → BPP	serviceConfirmedByFarmer: true, farmerPaymentMethod, farmerPaymentStatus: PAID
	System moves to FARMER_CONFIRMED → CLOSED	`on_update` + `status`	BPP → BAP	serviceStatus: FARMER_CONFIRMED → CLOSED
Cancellation	Farmer cancels before PROVIDER_ACCEPTED (always free)	`cancel`	BAP → BPP	cancellationReason, orderId

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
	Farmer cancels after PROVIDER_ACCEPTED (within cancellation window)	`cancel`	BAP → BPP	cancellationReason; BPP applies cancellationPolicy (refundable flag, window hours)
	Provider cancels (e.g., cannot service)	`cancel`	BPP → BAP	cancellationReason; farmer re-presented with alternatives
	Counterparty acknowledges	`on_cancel`	Opposite direction	serviceStatus: CANCELLED → CLOSED; refund details if applicable
Feedback / Rating	Farmer rates provider (1-5) + selects feedback option + optional text	`rate`	BAP → BPP	target: providerId, rating, selectedOption, additionalText (farmerFeedback)
	Provider rates farmer (cooperation, timeliness, payment)	`rate`	BPP → BAP	target: farmerId, rating, selectedOption, additionalText (providerFeedback)
	Acknowledgement	`on_rate`	Opposite direction	Feedback recorded confirmation
Support	Farmer requests provider contact / help	`support`	BAP → BPP	ref: orderId
	BPP returns contact info	`on_support`	BPP → BAP	contactInfo: phone / email / WhatsApp

4.6.2 Household Services — Complete Lifecycle

Example: School fee payment, hospital checkup, insurance enrolment, consumer goods delivery.

Structurally **identical** to Livelihood mapping (4.6.1). The Beckn endpoint sequence and semantics are the same. Category-specific differences are in the payload fields of `on_discover` / `on_select`:

- `HouseholdItemAttributes` replaces `LivelihoodItemAttributes`
- `subCategory` values: Education, Health, Insurance, Consumer Goods
- `serviceMode` enum: SERVICE_DELIVERY / PRODUCT_DELIVERY / FACILITY_VISIT / DIGITAL
- `beneficiaryType` (INDIVIDUAL / HOUSEHOLD / CHILD / ELDERLY) and `documentsRequired` may be populated

- Some services (e.g., school fee payment) may have `scheduledDate` as the payment deadline rather than a visit date

All lifecycle phases — discovery, selection, init, confirm, status, update, cancel, rate, support — apply the same way. The full Livelihood table above applies to Household with the above payload differences.

4.6.3 Market Produce — Complete Lifecycle (Reversed Economics)

Example: Farmer sells eggs, milk, farm produce to an aggregator / buyer. Money flows provider → farmer.

The Beckn endpoint sequence is the **same as Livelihood**. The semantic reversal is in the payload content, not in the protocol flow. Highlights:

Phase	Beckn Call	Semantic Reversal (vs Livelihood)
Discovery	`discover` / `on_discover`	Farmer advertises produce (`quantityAvailable` , `produceCategory`); BPP returns aggregators/buyers with `buying` prices and `collectionMode` (FARM_GATE_PICKUP / COLLECTION_POINT / FARMER_DELIVERS)
Selection	`select` / `on_select`	Quote is the **buying price per unit** aggregator offers for farmer's produce × quantity
Init / Confirm	`init` , `confirm` (and `on_*`)	Identical protocol, order created to pick up produce at farmer's location / collection point
Provider acceptance	`status` (push)	Aggregator accepts the collection request; serviceStatus: PROVIDER_ACCEPTED
Fulfilment	`status` / `update`	Aggregator collects produce, grades it (`acceptedGrades`), pays farmer; populates `providerPaymentReceived` to indicate *aggregator paid farmer* (NOT farmer paid provider). `amountReceived` = amount paid to farmer.
Farmer confirmation	`update`	Farmer confirms produce was collected and reports method received (Cash / M-Pesa); `farmerPaymentStatus: PAID` = farmer confirms they received payment
Cancellation	`cancel` / `on_cancel`	Either side can cancel before collection. After partial collection, use `update` to adjust quantity actually collected rather than full cancel.
Rate	`rate` / `on_rate`	Farmer rates aggregator's fairness (grading, price honoured, payment promptness); aggregator rates produce quality & farmer reliability
Support	`support` / `on_support`	Same contract as Livelihood

4.6.4 Advisory Content — Discover-Only Lifecycle

Example: Farmer requests advisory on poultry care, financial literacy, health & nutrition.

Advisory is **free, has no order, no payment, no scheduling**. Beckn endpoints used are limited to discovery and optional feedback.

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
Discovery with Google Content APIs + AI	Farmer selects Advisory → domain → category → sub-category	`discover`	BAP → BPP	Intent: category=Advisory-Content, subCategory (topic), farmerContext (location, enterprise, language)
	BPP calls **Google Content APIs** (per chosen domain — agri / health / market / finance) to retrieve curated content	*(BPP-internal; not a Beckn call)*	BPP ↔ Google Content API	Curated payload: articles, video URLs, structured guides, infographics
	BPP optionally calls **Google AI Interfaces** to personalise / summarise / translate content using farmer context	*(BPP-internal; not a Beckn call)*	BPP ↔ Google AI	Tailored content matched to farmer language, location, enterprise
	Final advisory delivered to farmer on WhatsApp	`on_discover`	BPP → BAP	Catalog with AdvisoryContentItemAttributes: contentType, contentUrl / inline text, contentDuration, targetAudience
Feedback (optional)	Farmer answers "Was this helpful?" [Yes] [No] or rates content	`rate`	BAP → BPP	target: contentId, rating (binary helpful/not or 1-5), optional additionalText
	Acknowledgement	`on_rate`	BPP → BAP	Feedback recorded; may trigger content improvement feedback loop
Support (rare)	Farmer asks for more info / clarification on content	`support` / `on_support`	BAP ↔ BPP	Contact info for advisory provider / content author (if applicable); typically not used in pilot

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
Not applicable	`select`, `init`, `confirm`, `status`, `update`, `cancel` — advisory has no order, so these endpoints are not used.			

Summary of endpoint coverage per flow:

Flow	discover	select	init	confirm	status	update	cancel	rate	support
Livelihood	✓	✓	✓	✓	✓	✓	✓	✓	✓
Household	✓	✓	✓	✓	✓	✓	✓	✓	✓
Market Produce	✓	✓	✓	✓	✓	✓	✓	✓	✓
Advisory	✓	—	—	—	—	—	—	✓	(rare)

4.7 Domain-Specific Flow Examples

The same 7-step flow engine powers agri and health. Below, the two exemplars side-by-side show that **only vocabulary and payload change** — the sequence, engine, and infrastructure are identical.

Step	Agri — Vaccination	Health — TB Screening
Need	Farmer's flock needs vaccination	Patient has persistent cough > 2 weeks
Search	`discover` near Naivasha	`discover` GeneXpert facility near Kisumu
Select	Pick John Kariuki (vet)	Pick Kisumu County Hospital
Confirm	Confirm 200 birds @ KES 30	Confirm referral, sputum collection date
Deliver	Vaccination performed	Sputum collected, sent for GeneXpert
Record	Provider notes 200 birds, healthy	Lab records test result (FHIR-compatible)
Outcome	Farmer confirms, payment, feedback	TB-positive → initiated on treatment; TB-negative → close

Flow F: TB Screening & Referral (Health domain)

```
Patient (WhatsApp): reports persistent cough via symptom triage
  --> webhook.php --> flow handler (template: health-services / screening-referral)
    Patient types location (free text, e.g. "Kisumu")
    --> Handler calls Google AI for location normalization
    <-- {county: "Kisumu", subCounty: "Kisumu Central"}
    --> Flow Engine: Search step
    --> BAP --> ONIX-BAP --> [Beckn Network] --> ONIX-BPP --> BPP
      --> Kuza Platform (query facilities by capability=GeneXpert + location)
    <-- on_discover: HealthScreeningItemAttributes (facility, capabilities, hours)
    Patient selects Kisumu County Hospital
  --> select / init / confirm --> order_id generated, sputum collection scheduled
  --> status(PROVIDER_ACCEPTED) --> WhatsApp "Visit on <date>. Bring ID."
  --> Deliver: patient visits, sputum collected
  --> Record: clinician enters test result via Kuza Web App --> status(COMPLETED)
  --> Outcome: HEALTH_OUTCOME_RECORDED
        result=POSITIVE --> auto-triggers treatment-adherence flow (Flow G)
        result=NEGATIVE --> close, advisory content on when to re-screen
```

Flow G: TB Treatment Adherence (Health domain)

```
Trigger: Flow F completes with result=POSITIVE (auto) OR clinician initiates manually
  --> Flow Engine instantiates template: health-services / treatment-adherence
  --> Seeker roles: patient (primary) + CHP (Provider for DOT supervision)

Daily loop (6-month regimen):
  Scheduler cron --> WhatsApp reminder to patient "Did you take today's dose?" [Yes]
[No]
    Patient [Yes] --> update(dose_taken=true) --> Record step
    Patient [No]  --> escalation --> CHP receives DOT visit task
                   --> CHP completes visit in Mobile Lite --> Record step
  Weekly:
    CHP confirms supervision visit, uploads signed DOT form
    Coordinator (SCTLC/CTLC) reviews adherence dashboard
  End of regimen:
    Outcome: CURED / TREATMENT_COMPLETED / LOST_TO_FOLLOWUP / FAILED
    --> HEALTH_OUTCOME_RECORDED event --> Analytics layer

Note: Same state machine, same BAP/BPP, same Kuza data store as any agri flow.
Only the flow template, capability tags, outcome vocabulary, and payload schemas differ.
```

5. Component Ownership

Component	Built By	Hosted On
Channel Handlers (WhatsApp, SMS/USSD, Mobile Lite) — incl. `beckn_marketplace_handler.php`	Kuza	GCP
Flow Engine (7-step state machine + flow templates)	Kuza	GCP
BAP application logic (agri + health)	Kuza	GCP
BPP application logic (agri + health)	Kuza	GCP
Kuza Web App (provider interface)	Kuza	GCP
Provider WhatsApp notifications	Kuza	GCP
ONIX Adapters (BAP + BPP)	Beckn (open source, Go)	GCP (Docker)
DeDi Registry	Beckn	Beckn Fabric (external)
Catalog Service	Beckn	Beckn Fabric (external)
Discovery Service	Beckn	Beckn Fabric (external)
Google Cloud Platform (GCP) — hosting infrastructure	Google	GCP
Google Content APIs (curated content for agri / health / market / finance)	Google	GCP
Google AI Interfaces (location normalization, content tailoring, future intent extraction)	Google	GCP
MySQL database (with PostgreSQL migration in flight via `kuza-next`)	Kuza	GCP (in-country)
Analytics & Intelligence layer (dashboards, ML models, surveillance)	Kuza	GCP

6. Database Schema

The following schema additions are required on MySQL 8 (GCP). These tables live alongside existing Kuza Platform tables and reference the existing `user` table via foreign keys.

Entity Relationship Summary

```
user (existing)
  |-- beckn_farmer_profile      (1:1, farmer-specific fields)
  |-- beckn_provider_catalog    (1:N, provider's services)
  |-- beckn_provider_service_areas (1:N, coverage areas)

beckn_subcategory_master        (standalone taxonomy)

beckn_orders
  |-- beckn_order_status_history (1:N, audit trail)
  |-- beckn_payments             (1:N, farmer + provider payments)
  |-- beckn_feedback             (1:N, ratings from both parties)

beckn_conversation_state        (keyed by phone + session)
```

Key Table Definitions

`beckn_orders`

```
order_id          BIGINT AUTO_INCREMENT PRIMARY KEY
order_uuid        VARCHAR(36) UNIQUE NOT NULL
beckn_order_id    VARCHAR(255)        -- Beckn network order ID
farmer_user_id    INT                 -- FK to user.id
provider_user_id  INT                 -- FK to user.id
category          VARCHAR(100)        -- livelihood | household | market | advisory
sub_category      VARCHAR(255)
service_name      VARCHAR(255)
quantity          DECIMAL(10,2)
unit              VARCHAR(50)
quoted_price      DECIMAL(10,2)
currency          VARCHAR(10) DEFAULT 'KES'
status            ENUM('ORDER_PLACED','PROVIDER_ACCEPTED','PROVIDER_REJECTED',
                       'IN_PROGRESS','COMPLETED','FARMER_CONFIRMED',
                       'CANCELLED','CLOSED')
farmer_location   VARCHAR(255)        -- normalized county/sub-county
provider_location VARCHAR(255)
payment_method    VARCHAR(50)         -- cash | mpesa
created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
partner_id        VARCHAR(50)         -- multi-tenancy scope
```

`beckn_conversation_state`

```
id                BIGINT AUTO_INCREMENT PRIMARY KEY
phone_number      VARCHAR(20) NOT NULL
```

```
  session_id        VARCHAR(100)
  current_flow      ENUM('livelihood','household','market','advisory')
  current_step      VARCHAR(100)
  state_data        JSON                -- accumulated selections, context
  beckn_txn_id      VARCHAR(255)        -- Beckn transaction correlation
  last_activity     TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  partner_id        VARCHAR(50)
  INDEX idx_phone_session (phone_number, session_id)
```

7. Deployment Topology

All Kuza-built components are deployed on Google Cloud Platform (GCP), in-country to preserve **data sovereignty** for patient and farmer records. Beckn Fabric services are externally hosted and accessed over HTTPS.

> **Health-domain deployment note**
>
> For health-services deployments, the Kuza Platform data store exposes a **FHIR-compatible** export adapter so clinical records can be synchronised with MoH systems without reshaping. Audit trail (who changed what, when) is retained on every record, and seeker / patient PII never leaves the in-country GCP region.

GCP Instance Layout

Component	Runtime	Port	Notes
Kuza Platform (PHP)	Apache + PHP 7.4	443	Existing platform + WhatsApp Handler + BAP + BPP
Kuza Web App	Nginx (static)	443	Ionic Angular build served as static files
ONIX-BAP	Docker (`fidedocker/onix-adapter`)	5001	BAP-side adapter configuration
ONIX-BPP	Docker (`fidedocker/onix-adapter`)	5002	BPP-side adapter configuration
Google Content APIs	Cloud Run / API Gateway	443	Curated content per domain (agri / health / market / finance) for chatbot delivery
Google AI Interfaces	Cloud Functions / Vertex AI / API	443	Location normalization, content tailoring (summarisation, translation, personalisation)

Component	Runtime	Port	Notes
MySQL 8	MySQL Server	3306	Single database instance

External Dependencies

Service	Endpoint	Protocol
WhatsApp Cloud API	`graph.facebook.com`	HTTPS REST
DeDi Registry	`fabric.nfh.global/registry/dedi`	HTTPS REST
Beckn Catalog Service	`fabric.nfh.global/beckn/catalog`	HTTPS REST
Beckn Discovery Service	`fabric.nfh.global`	HTTPS REST (Beckn v2)

Deployment Note

ONIX adapters run as Docker containers on the same GCP instance. They require outbound HTTPS access to Beckn Fabric endpoints and inbound access from the BAP/BPP application logic on localhost.

8. Domain-Shared vs Domain-Specific

A clear separation between what is *shared across all domains* and what is *bespoke per domain* is what makes adding the next domain (e.g., education, finance, agri-insurance) a matter of configuration rather than re-architecture.

Concern	Domain-Shared (one implementation)	Domain-Specific (per domain)
Flow orchestration	7-step Flow Engine, state machine, transaction correlation	Flow templates (vaccination, screening, treatment-adherence)
Beckn protocol	BAP / BPP / ONIX / Network / Registry / Catalog / Discovery	Beckn domain code (`local-services` vs `health-services`); extension schemas (`LivelihoodItemAttributes` vs `HealthScreeningItemAttributes`)
Channels	WhatsApp, Web App, SMS/USSD, Mobile Lite	— (no variation)

Concern	Domain-Shared (one implementation)	Domain-Specific (per domain)
Data layer	Kuza Platform (MySQL / PG), audit trail, multi-tenancy, partner scoping	Record schemas (FHIR-compatible for health; none required for agri); regulatory / compliance fields
Identity & actors	Seeker / Provider / Coordinator / Validator / Sponsor abstraction; UUID spine	Role vocabulary (farmer / patient; agripreneur / clinician / lab / CHP)
Matching	Service-area + capability-tag matching engine	Capability vocabulary (vet, agrovet vs clinician, GeneXpert, X-ray)
Google services	GCP hosting, AI location normalization, AI tailoring	Content API endpoint per domain (agri / health / market / finance)
Analytics	Event / outcome ingestion pipeline; 3-horizon model	Outcome definition (harvest yield vs cure / mortality); KPIs; dashboards
Master data	Counties, sub-counties, wards (shared across domains)	Sub-category taxonomies; outcome vocabularies

9. Architectural Principles

Seven principles guide the design and evolution of the platform. They originate in TIBU (the health deployment) and have been generalised to apply to all domains:

1. **Preserve, don't replace** — existing data systems (Kuza MySQL, TIBU registers, MoH systems) stay; the platform adds orchestration on top. No rip-and-replace.
2. **UUID is the spine** — every entity (seeker, provider, order, flow instance, outcome) has a universal UUID so records can be linked across systems, domains, and time.
3. **Channel-agnostic intelligence** — the same decision / flow step can be delivered via WhatsApp, Web, SMS/USSD, or Mobile Lite. Channel is a rendering concern, not a logic concern.
4. **Domain-agnostic infrastructure** — the same Flow Engine handles agri and health. Adding a domain is a configuration of templates, schemas, and taxonomies — not new infrastructure.
5. **Sovereign data** — patient / farmer data stays in country, with full audit trail. Schemas are compliant (FHIR-compatible for health; auditable for agri).
6. **Progressive disclosure** — UI complexity matches user role. A seeker sees a short prompt; a coordinator sees a dashboard; a validator sees an audit view. Same underlying data, different surfaces.

7. **Flow Engine is optional until proven** — analytics (Horizon 1) can run without it. The engine earns its place as outcome data accumulates and ML (Horizon 2-3) becomes viable.